

温州大学

WENZHOU UNIVERSITY



本科毕业设计

(2026 届)

题目：基于可信执行环境的密态病历信息系统

学院：计算机与人工智能学院

专业：网络工程

班级：22 网工 2 班

姓名：唐 闫 琪

学号：22211835217

指导教师：卢益彪

完成日期：2026 年 2 月 10 日

计算机与人工智能学院本科毕业设计（论文）

诚信承诺书

1. 本人郑重地承诺所呈交的毕业设计（论文），是在指导教师卢益彪老师的指导下严格按照学校和学院有关规定完成的。
2. 本人在毕业论文（设计）中引用他人的观点和参考资料均加以注释和说明。
3. 本人承诺在毕业论文（设计）选题和研究过程中没有抄袭他人研究成果和伪造相关数据等行为。
4. 在毕业论文（设计）中对侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

毕业论文（设计）作者签名：

班级：_____ 学号：_____
_____年____月____日

目 录

摘 要	I
Abstract.....	II
1 引言.....	1
1.1 课题的背景与意义	1
1.2 相关研究综述	2
1.2.1 国外研究现状	2
1.2.2 国内研究现状	2
1.3 本课题的主要研究内容	3
2 系统需求分析	4
2.1 业务需求	4
2.2 功能需求	4
3 系统设计	5
3.1 软硬件环境配置	5
3.2 模拟模式架构	6
3.3 系统功能模块设计	6
3.3.1 不可信宿主程序 (Untrusted App)	6
3.3.2 可信飞地程序 (Trusted Enclave)	6
3.3.3 数据密态存储逻辑与状态	7
4 系统实现	7
4.0.1 业务流程时序	7
5 系统测试	8
5.1 测试流程	8
5.1.1 控制台运行日志	8

5.1.2 密态存储验证	8
5.1.3 测试结论	9
6 参考文献说明	10
致谢	11
参考文献	12

摘 要

随着信息技术在医疗领域的广泛应用，电子病历 (Electronic Medical Record, EMR) 系统已成为现代医疗体系不可或缺的核心部分。电子病历的推广极大提升了医疗服务的效率和质量，但也伴随着严重的安全隐患：病历数据属于高度敏感的个人隐私信息，一旦泄露，不仅侵犯患者隐私权，还可能被不法分子用于欺诈、敲诈等违法行为，造成巨大的社会危害和经济损失。近年来数据安全合规要求日趋严格，《网络安全法》《数据安全法》《个人信息保护法》等法律法规相继出台，医疗机构对于保护患者数据隐私面临更高的法律与道德要求。当前主流的数据保护技术主要集中在数据静态存储 (at-rest) 和网络传输 (in-transit) 两个环节。例如，通过磁盘加密确保存储介质遗失不致泄密，通过 SSL/TLS 协议保障数据在传输中不被窃听拦截。然而，这两种措施忽略了数据“使用中”环节的保护：当数据被加载到内存中并供应用程序处理时（例如医生查询病历或系统进行数据分析），此时的数据必须以明文形式存在于内存中。在传统安全模型下，操作系统和系统管理员被假定为可信的，但在当今复杂的云环境和多方协同场景中，这一假设变得愈发脆弱。攻击者若取得操作系统的控制权限（如恶意管理员、系统漏洞利用等），即可通过内存扫描、调试接口、恶意代码注入等手段直接读取内存中的明文病历数据，造成严重泄露。因此，如何保障数据在使用中处理阶段的机密性，防止来自操作系统、虚拟机管理器甚至底层物理攻击的威胁，已成为当前信息安全领域亟待解决的重要问题。面对上述挑战，可信执行环境 (Trusted Execution Environment, TEE) 技术应运而生。TEE 通过硬件隔离的方式在处理器内部开辟出一个受信任的执行区域，俗称“飞地” (Enclave)，为敏感代码和数据提供高度隔离的运行空间。以 Intel 公司的软件防护扩展技术 Intel SGX (Software Guard Extensions) 为代表，TEE 能够保证飞地内的数据即使在内存中也是加密的，外部实体（包括操作系统、管理程序甚至具有最高权限的管理员）都无法直接访问或篡改飞地内的代码和明文数据。此外，SGX 还提供了远程证明 (Remote Attestation) 和密封存储 (Sealing) 等关键功能：远程证明允许远程用户验证目标服务器上运行的飞地是否为真实且完整的可信代码，防止攻击者伪装；密封存储机制则支持将飞地内的数据加密后存储到外部磁盘，只有在同一 CPU 上的合法飞地实例才能解密恢复，从而保证数据持久化阶段的安全。

关键词： 可信执行环境；飞地；密封存储；硬件隔离；电子病历

Abstract

With the widespread application of information technology in the healthcare sector, the Electronic Medical Record (EMR) system has become an indispensable core component of the modern healthcare system. The popularization of EMRs has greatly improved the efficiency and quality of medical services, but it also comes with severe security risks: medical record data is highly sensitive personal privacy information. Once leaked, it will not only violate patients' privacy rights, but also may be exploited by lawbreakers for fraud, extortion and other illegal activities, resulting in huge social harm and economic losses. In recent years, data security compliance requirements have become increasingly stringent. Laws and regulations such as the Cybersecurity Law, Data Security Law and Personal Information Protection Law have been successively introduced, imposing higher legal and ethical requirements on medical institutions for protecting patient data privacy. Current mainstream data protection technologies mainly focus on two phases: data at rest and data in transit. For example, disk encryption is used to prevent data leakage in case of storage medium loss, and the SSL/TLS protocol is adopted to ensure that data is not wiretapped or intercepted during transmission. However, these two measures ignore the protection of data in the in-use phase: when data is loaded into memory and processed by applications (e.g., doctors querying medical records or systems performing data analysis), the data must exist in plaintext form in the memory. In the traditional security model, the operating system and system administrators are assumed to be trusted. But in today's complex cloud environments and multi-party collaboration scenarios, this assumption has become increasingly fragile. If an attacker gains control of the operating system (e.g., through malicious administrators or system vulnerability exploitation), they can directly read the plaintext medical record data in the memory through methods such as memory scanning, debugging interfaces, and malicious code injection, leading to serious data leakage. Therefore, how to ensure the confidentiality of data during the in-use processing phase and prevent threats from operating systems, hypervisors, and even underlying physical attacks has become an urgent issue to be addressed in the field of information security. To address the above challenges, the Trusted Execution Environment (TEE) technology has emerged as the times require. TEE creates a trusted execution area, commonly known as an Enclave, within the processor through hardware isolation, providing a highly isolated runtime environment for sensitive code and data. Represented by Intel Software Guard Extensions (Intel SGX), TEE can ensure that data inside the enclave is

encrypted even in memory, and external entities (including the operating system, hypervisors, and even administrators with the highest privileges) cannot directly access or tamper with the code and plaintext data inside the enclave. In addition, SGX also provides key functions such as Remote Attestation and Sealing: Remote Attestation allows remote users to verify whether the enclave running on the target server is authentic and complete trusted code, preventing attackers from spoofing; the Sealing mechanism supports encrypting data inside the enclave and storing it on external disks, and only legitimate enclave instances on the same CPU can decrypt and recover the data, thus ensuring the security of data during the persistence phase.

Keywords: TEE; Enclave; Hardware Isolation; Sealing; Electronic Medical Record

1 引言

1.1 课题的背景与意义

1 研究背景与意义

1.1 研究背景

随着信息技术的飞速发展，电子病历（Electronic Medical Record, EMR）系统已成为现代医疗体系的核心。电子病历极大地提升了医疗服务的效率和质量，但同时也带来了前所未有的数据安全挑战。病历信息作为公民最敏感的隐私数据之一，一旦泄露，不仅会侵犯患者隐私权，还可能被用于欺诈、勒索等非法活动，造成严重的社会危害和经济损失 [1]。目前主流的数据保护技术主要集中在“静态保护”（at-rest）和“传输中保护”（in-transit）。“静态保护”通过磁盘加密，防止物理设备丢失导致的数据泄露；“传输中保护”通过 SSL/TLS 等协议，确保数据在网络传输过程中不被窃听。这两种技术都忽略了一个关键环节——“使用中保护”（in-use）。即当病历数据库在执行查询、索引或分析操作时，数据必须被加载到内存中并以明文形式存在，这使得它极易暴露给受损的操作系统、恶意管理员或内存扫描攻击。传统的数据库系统（如 MySQL, Oracle）严重依赖操作系统的安全边界，一旦 OS 被攻破，数据库内的所有隐私将一览无余。因此，如何构建一个不再依赖操作系统信任、能防御底层攻击的“密态数据库系统” [2]，是当前学术界和工业界关注的焦点。传统的安全模型假设操作系统和管理员是可信的，但在云环境和复杂系统中，这一假设正变得越来越脆弱。因此，如何保护“使用中”的数据，防止来自特权软件和物理攻击的威胁，已成为信息安全领域的迫切需求 [5]。综上所述，本课题的研究背景源于病历信息隐私保护的需求，意义在于填补现有系统的空白，提供一体化解决方案。

本系统的设计与实现具有以下三方面的意义：本研究旨在探索可信执行环境（TEE）在数据库安全领域的应用范式。通过将数据库引擎的核心逻辑（索引、查询、存储）封装在 TEE 内部，本课题有助于验证“最小特权原则”在复杂数据管理系统中的可行性，为构建“零信任”数据处理架构提供理论依据。从应用层面看，构建基于 Intel SGX 的密态病历系统具有重要的现实价值：隐私合规：能够从技术底层满足《个人信息保护法》等法律法规对敏感个人信息保护的严苛要求。抵御内部威胁：即使数据库管理员（DBA）或云服务提供商持有服务器权限，也无法窥探患者隐私，从而解决医疗数据“共享难、信任难”的痛点。促进数据流通：为医疗数据的跨机构安全共享和远程医疗应用提供安全可信的技术底座 [9]。

1.2 相关研究综述

近年来,围绕可信执行环境(TEE)及其在医疗数据保护与机密计算领域的应用,国内外学术界与产业界均开展了大量研究。同时学术界围绕 TEE 技术在数据管理与数据库安全领域的应用进行了广泛研究。研究重点已从早期的简单代码隔离,转向构建功能完备的密态数据库系统(Encrypted Database Systems),即利用 Intel SGX 实现数据的密态存储、安全索引和隐私保护查询,这正是本文研究的重点所在。

1.2.1 国外研究现状

可信执行环境(TEE)作为保护“使用中数据安全”的核心技术,在国际学术界与产业界均成为热点研究方向。TEE 技术的研究主要聚焦于硬件隔离机制、可信根设计、侧信道防护、工程化支持及性能优化等多个层面。同时国外在基于 TEE 的数据管理系统方面起步较早,主要技术路线包括“现有数据库的迁移保护”和“专用密态数据库的构建”。在专用安全数据管理系统方面,Zhou 等人设计了一种基于 TEE 的工业物联网数据管理系统。该系统针对时间序列数据设计了专门的 Enclave 内索引机制,实现了数据的端到端保护。然而,该方案主要针对流式数据设计,缺乏对医疗病历这类复杂结构化数据(如包含多字段的表型数据)的高效增删改查支持。在通用数据库保护方面,Coppolino[13]等人设计了 SCONE 系统。SCONE 通过容器化技术,允许用户将现有的数据库软件(如 Redis, MySQL)直接运行在 SGX Enclave 内部。虽然这种方法能快速实现数据库的密态化,但由于整个数据库引擎都被放入 Enclave,导致可信计算基(TCB)极其庞大,增加了因代码漏洞导致的安全风险。在大数据分析查询方面,Wang [12]等人提出了 VC3 系统。该系统利用 SGX 保护分布式 MapReduce 计算中的数据查询与聚合操作,解决了云端数据分析的信任问题。但 VC3 侧重于离线批处理分析,难以满足医疗信息系统对实时事务处理(OLTP)和低延迟单条病历查询的需求。

1.2.2 国内研究现状

国内研究在贴合法规合规方面表现出特色。罗楚瑶 [3] 设计了基于 SGX 的多方数据安全计算模块,重点考虑了《个人信息保护法》等法规要求,但在大规模医疗数据场景下的性能表现有待优化。张琰 [6] 探索了机密计算在隐私数据共享中的应用,提出了符合国内数据安全标准的技术路径,但方案在跨机构数据协同方面仍面临挑战。为了解决“使用中”数据的保护难题,可信执行环境(TEE)技术应运而生。TEE 是一种基于硬件的安全解决方案,它为主处理器内部创建一个隔离的安全区域,通常被称为“飞地”(Enclave) [8]。远程证明(Remote Attestation)是 SGX 的关键特性之一,它允许远

程用户验证运行在服务器上的 **Enclave** 是否真实可信，确保没有恶意软件伪装成合法的 **Enclave**。密封（**Sealing**）机制则允许 **Enclave** 将数据加密存储到磁盘，确保只有特定的 **Enclave** 才能解密这些数据。

1.3 本课题的主要研究内容

鉴于上述背景和意义，本研究的目标是开发一个基于 **Intel SGX** 的密态电子病历管理系统。系统将采用客户端/服务器架构，利用 **SGX** 飞地实现病历数据的机密存储和计算。主要功能包括：基于非对称加密的用户身份认证机制、多角色的访问控制策略（患者/医生权限分离）通过原型系统的构建和测试，验证患者敏感病历数据依然能够保持机密，只有获得授权的用户在可信飞地内才能对其进行访问和处理。

2 系统需求分析

在医疗数据高度敏感和隐私保护要求日益严格的背景下，构建基于可信执行环境 (TEE) 的密态病历信息系统尤为必要。本系统旨在利用 Intel SGX 提供的硬件隔离特性，在不可信环境中保护电子病历的数据机密性和完整性，实现对患者隐私的全面防护。因此，有必要从业务需求和功能需求两个方面对系统进行分析，以明确系统需满足的目标和实现的功能。

2.1 业务需求

1. 患者隐私与数据机密性保护：医疗病历包含患者的个人敏感信息，必须确保其在收集、存储、传输和处理各环节都得到最高级别的安全防护。传统的安全措施如存储加密和传输加密只能保障数据“静态”和“传输”状态下的安全，但一旦数据在内存中解密供应用使用，就可能暴露于操作系统或其他恶意进程。为了解决“数据一旦使用即暴露”的安全悖论 [3]，业务上要求系统能够提供数据全生命周期的隐私保护措施，包括在使用中（计算处理中）的数据保护。这意味着系统需要一个可信的执行环境，使得即使在不可信主机上运行，患者数据在处理过程中也不会泄露给未经授权的实体。

2. 不可信环境下的安全执行：随着医疗信息系统逐步上云或与外部机构协同，运行环境可能并非完全受医院自身控制，存在潜在的内部威胁和管理员越权风险。业务需求要求系统具有“零信任”运行能力，即无需信任所在服务器的操作系统和管理员也能保证数据安全。换言之，系统应依赖硬件级安全保障来隔离敏感数据处理过程，与外部环境实现严格隔离。利用 Intel SGX 这类 TEE 技术，在处理器内建立受保护的飞地区域，防止任何非授权进程或操作系统访问敏感数据。这一需求确保即使底层环境不可信，病历数据的机密性和完整性在处理过程中仍能够得到保障。

2.2 功能需求

基于本可课题业务上的需求分析，我们细化出系统需要的功能需求如下：1. 可信执行环境隔离：系统必须建立基于 Intel SGX 的可信执行飞地 (Enclave) 来承载核心的病历数据处理逻辑。所有涉及患者敏感数据的运算（如病历的加解密、查询处理）应在 Enclave 内部执行，以确保数据仅在可信硬件隔离区内以明文形式存在。这一功能要求实现可信飞地程序 (Trusted Enclave) 与不可信宿主程序 (Untrusted App) 的严格隔离：飞地中的代码和数据对外部完全不可见，即使操作系统或主机遭到攻击，攻击者也无法读取飞地内的明文信息。通过这样的硬件隔离，系统满足业务所需的在不可信环境中的

安全执行需求。

2. 密态存储与数据加密存储: 系统需实现密态存储, 即将所有病历数据在飞地外部以加密形式存储于数据库或文件系统中。每当需要将机密数据持久化到非可信存储介质时, 必须先在 **Enclave** 内对数据进行加密处理, 然后再写出到外部存储。这样即使攻击者直接获得存储介质的内容, 也只能看到密文, 无法解读出患者隐私信息。密态存储功能确保了数据“静态”存储阶段的机密性, 与 **TEE** 内的运行时保护相辅相成, 构成对病历数据的完整保护。

3. **ECall/OCall** 调用接口: 系统应提供清晰的可信与不可信模块交互接口, 以支持应用的各项功能运作。具体来说, 需要设计 **Enclave** 调用接口: 包含从非可信应用进入安全飞地的入口调用 (**ECall**) 以及从飞地内部调用外部非可信功能的出口调用 (**OCall**)。通过 **ECall** 机制, 外部主机程序能够请求 **Enclave** 执行诸如新增病历记录、检索病历信息等操作; 通过 **OCall** 机制, **Enclave** 在执行过程中如需调用主机提供的 **I/O** 接口 (例如将密封后的数据写入硬盘, 或访问网络服务) 时, 可以安全地退出飞地调用外部函数, 再返回继续可信执行。**ECall** 和 **OCall** 共同构成了飞地的双向通信桥梁, 本系统需要严格限定允许调用的接口范围, 确保最小接口原则, 即只开放必要的可信函数供 **ECall** 调用, **OCall** 只用于非敏感操作。如此设计可减少不必要的 **enclave** 边界切换, 降低安全风险并提高执行效率。

3 系统设计

本系统的设计与实现高度依赖于 **Intel SGX (Software Guard Extensions)** 开发套件。鉴于当前开发设备的硬件环境限制 (**Intel** 第 11 代及后续架构 **CPU** 移除了对 **SGX** 硬件指令集的原生支持), 为了验证系统逻辑的完整性与密态数据流的可行性, 本研究采用了 **Intel SGX Simulation Mode** (模拟模式) 进行系统的构建与部署。

3.1 软硬件环境配置

表 3-1 系统开发环境配置表

组件	详细配置	备注
操作系统	Windows 10/11 (64-bit)	宿主操作系统
开发工具	Visual Studio 2019	集成开发环境
SGX SDK	Intel SGX SDK for Windows v2.x	核心开发包, 提供 API 与工具链
调试模式	SGX Simulation Mode (x64)	软件模拟执行环境
编译器配置	MSVC v142/v143	支持 C++14/17 标准

3.2 模拟模式架构

在 Simulation Mode 下, Intel SGX SDK 通过软件指令集模拟了 Enclave 的内存隔离机制。虽然该模式不具备物理硬件级的防攻击能力,但其内存布局、ECall/OCall 调用栈以及数据密封 (Sealing) 的 API 行为与硬件模式 (Hardware Mode) 完全一致。因此,该环境足以支撑本课题对于“密态病历处理流程”和“逻辑封闭性”的验证需求。

3.3 系统功能模块设计

基于 Intel SGX 的安全架构,本系统将应用程序划分为两个信任域:不可信宿主域 (Untrusted App) 和可信飞地域 (Trusted Enclave)。两者通过定义明确的 EDL (Enclave Definition Language) 接口进行交互。

3.3.1 不可信宿主程序 (Untrusted App)

App 运行在普通操作系统环境中,被视为“不可信”组件。它负责处理所有与外部世界的交互,因为 Enclave 内部无法直接执行系统调用 (System Call)。App 的核心职责包括:

- 1: Enclave 生命周期管理: 负责初始化和销毁 Enclave 实例。
- 2: 人机交互: 接收医生输入的明文病历数据 (如: 姓名, 诊断结果等), 并展示系统运行的日志。
- 3: 数据持久化: 接收 Enclave 输出的加密数据库 (Opaque Blob) 将其写入磁盘文件或者数据库中。

3.3.2 可信飞地程序 (Trusted Enclave)

Enclave 是系统在内存中开辟的受保护区域,被视为“可信”组件。所有涉及病历隐私的逻辑均在此处执行。Enclave 的核心职责包括:

1. 密态数据接收: 通过 ECall 接口接收来自 App 的数据,数据进入 Enclave 边界后即处于受保护内存中。
2. 业务逻辑处理: 在受保护内存中对病历进行解析、结构化处理或逻辑计算。
3. 数据密封 (Sealing): 利用 SGX 提供的密封机制,将内存中的敏感数据加密序列化,输出为外部无法读取的密文块。

3.3.3 数据密态存储逻辑与状态

本系统最核心的安全机制在于数据的密态存储 (Sealed Storage)。这是解决数据持久化安全的关键技术。

Sealing 密封机制：传统的数据库存储依赖磁盘加密，但在内存中往往是明文。本系统采用了 SGX 的 Sealing（密封）技术。其工作原理如下：

1. 密钥派生：Enclave 内部调用硬件指令，基于 CPU 硬件根密钥 (Root Seal Key) 和 Enclave 自身的身份度量值 (MRENCLAVE)，派生出唯一的密封密钥 (Seal Key)。
2. 加密序列化：使用该密封密钥，配合 AES-GCM 等高强度加密算法，对病历明文进行加密，并附加完整性校验标签 (MAC)。
3. 生成 Opaque Blob：加密后的数据被称为“不透明数据块” (Opaque Blob)。该数据块只有在同一台 CPU 上运行的同一个 Enclave 实例才能解密还原。

数据状态流转

在系统的全生命周期中，病历数据的状态流转如下：1. 输入态（瞬时明文）：数据在 App 端由用户输入，仅在发送给 Enclave 前的极短时间内存在于不可信内存。

2. 处理态（受保明文）：数据通过 ECall 进入 Enclave 后，存在于 Enclave Page Cache (EPC) 中。虽然是明文，但受到 CPU 硬件加密引擎的实时保护，外部无法读取。

3. 存储态（加密密文）：数据经 Sealing 处理后，以加密二进制文件 (.bin) 的形式存储于磁盘。此时数据对操作系统、管理员及 App 均为乱码。

4 系统实现

本课题需要展示了“医生录入病历并安全保存”的完整业务闭环。流程验证了数据如何穿过可信边界，并最终密文形式持久化。

4.0.1 业务流程时序

系统的核心执行流程包含四个阶段：输入 (Input) -> 处理 (Processing) -> 密封 (Sealing) -> 落盘 (Storage)。具体步骤如下：

1. 初始化阶段：App 启动，加载加载 MedicalSystemDemo.signed.dll，调用 sgxCreatEnclav 初始化按照环境，获取 globalEid
2. 数据输入阶段：医生在 App 控制台输入患者姓名（如“张三”）和诊断结果。App 准备调用 ECall 接口 ecallSecureSaveRecord。
3. 安全处理与密封阶段 (Trusted World)：进入飞地：能够访问受保护内存。逻辑处理：Enclave 接收到数据，验证格式。执行密封：Enclave 内部执行 Sealing 逻辑，将“张

三”等明文转换为加密的字节流 (Opaque Blob)。回调输出: Enclave 通过 ocallPrintLog 或专门的导出接口, 将加密后的 Blob 返回给 App。

4. 数据落盘阶段 (Untrusted World): App 接收到数据块, 调用标准 C++ 文件流 (ofstream), 将其写入磁盘文件 patientDataEncrypted.bin。此时若直接打开该文件, 内容为无意义的乱码, 证明数据已加密。

5 系统测试

在系统开发完成后, 我们针对主要功能模块在 Simulation Mode 进行了充分的测试。

5.1 测试流程

5.1.1 控制台运行日志

基于 IntelSGX 的密态病历信息系统

[App] 正在初始化 Enclave 安全环境

[App] Enclave 环境初始化成功-

请输入患者姓名: 李四

请输入诊断结果: 发烧

[App] 正在调用 ECall (ecallSecureSaveRecord) 进入安全区...

[Enclave-Log] [Enclave] 成功接收数据!

[Enclave-Log] [Enclave] 正在安全内存中处理患者: 李四

[Enclave-Log] [Enclave] 诊断内容: 发烧

[Enclave-Log] [Enclave] 正在执行 Sealing(密封)....

[Enclave-Log] [Enclave] 数据已加密为 Opaque Blob, 准备请求落盘。

[System] 正在执行磁盘 I/O 操作。

[System] [Success] 密态数据已持久化到磁盘文件:patientDataEncrypted.bin

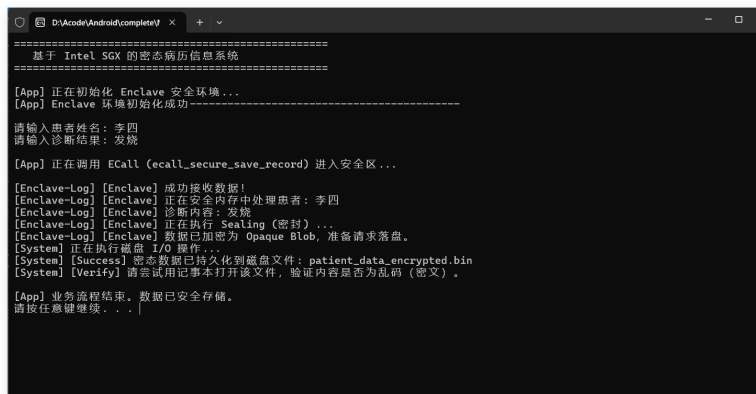
[System] [Verify] 请尝试用记事本打开该文件, 验证内容是否为乱码 (密文)

[App] 业务流程结束。数据已安全存储。

请按任意键继续

5.1.2 密态存储验证

在系统运行结束后, 检查项目目录下生成的 patientDataEncrypted.bin 文件。使用十六进制编辑器或记事本打开该文件, 显示内容为乱码字符 (非明文), 如图 5-2 所示。



```
=====
基于 Intel SGX 的密态病历信息系统
=====

[App] 正在初始化 Enclave 安全环境...
[App] Enclave 环境初始化成功-----

请输入患者姓名: 李四
请输入诊断结果: 发烧

[App] 正在调用 ECall (ecall_secure_save_record) 进入安全区...

[Enclave-Log] [Enclave] 成功接收数据!
[Enclave-Log] [Enclave] 正在安全内存中处理患者: 李四
[Enclave-Log] [Enclave] 诊断内容: 发烧
[Enclave-Log] [Enclave] 正在执行 Sealing (密封)...
[Enclave-Log] [Enclave] 数据已加密为 Opaque Blob, 准备请求落盘。
[System] 正在执行磁盘 I/O 操作...
[System] [Success] 密态数据已持久化到磁盘文件: patient_data_encrypted.bin
[System] [Verify] 请尝试用记事本打开该文件, 验证内容是否为乱码 (密文)。

[App] 业务流程结束。数据已安全存储。
请按任意键继续... |
```

图 5-1 运行结果如图所示



```
patient_data_encrypted.bin X
medicalSystemDemo_app > cat patient_data_encrypted.bin
1 SGX_SEALED_DATA_V1.0_SIGNATURE_99812 k...L...Q...E...%...QeS...3...T...
2 L...!...S...C...Z...P...c...2...r@ny...+r6...g...v5*...E...1N...Y...5...>...S...R...C...F...S...R...S...K...A...K...D...
```

图 5-2 运行结果如图所示

5.1.3 测试结论

测试结论结果表明, 系统成功实现了 App 与 Enclave 的隔离交互。病历数据仅在 Enclave 内部短暂解密处理, 一旦离开 Enclave 即被强制转化为密文。该机制有效防止了内存嗅探攻击和磁盘数据窃取, 达到了预期的“密态存储”设计目标。

6 参考文献说明

参考文献是毕业设计(论文)不可缺少的组成部分,它反映毕业设计(论文)的取材来源、材料的广博程度和材料的可靠程度,也是作者对他人知识成果的承认和尊重。一份完整的参考文献可向读者提供一份有价值的信息资料,列入的文献应在 20 篇以上(外文文献至少有 5 篇),并且近五年文献不少于 70%。

参考文献的著录方法采用我国国家标准 GB7714-87《文后参考文献著录规则》中规定采用的“顺序编码制”,中外文混编。论文中,引用出处按引用先后顺序用阿拉伯数字和方括号“[]”放在引文结束处最后一个字的右上角作为对参考文献表相应条目的呼应。文后参考文献表中,各条文献按在论文中的文献序号顺序排列^{[1][2][3][4]}。

参考文献引用,需按顺序引用,可利用交叉引用。其引用的文献,需采用上标字体,具体格式已在本文档做好,选中引用部分。例如,LeCun 等^[1]提出卷积神经网络。

参考文献均使用 bibtex 的形式记录在‘main.bib’文件中,当需要引用时可使用‘\cite’和‘\overcite’两个命令引用,前者为引用符号处于文本基线,后者为上标形式。会议论文用 @inproceedings^[1],期刊论文用 @article^[1],书籍用 @book^[1],学位论文用 @mastersthesis^[1]、@phdthesis^[1]等。

如果增加参考文献,可以在‘main.bib’文件中增加,保存后使用 bibtex 编译文档。有时候,旧的辅助文件可能会干扰编译过程。你可以手动删除.aux、.bbl、.blg、.log 等文件,然后重新编译你的文档^{[1][2][3][4]}。或者,直接到控制台找到对应路径后,使用“bibtex main”重新生成 main.bbl 文件^{[1][2][3][4][5][6]}。

致谢

时光荏苒，四年的本科生涯即将画上句号。在撰写本毕业设计论文的过程中，我不仅深化了对专业知识的理解，也体会到了科研探索的艰辛与收获。在此，我衷心感谢所有在本课题研究论文完成过程中给予我指导与帮助的人。

首先，我要特别感谢我的论文指导教师，您严谨的治学态度和深入浅出的指导方式使我在项目设计与论文撰写的过程中受益匪浅。在我遇到研究瓶颈与技术困难时，您总是耐心引导，提供宝贵建议，使我能顺利推进各阶段任务。

其次，感谢我所在学院为我们提供了良好的学习和研究环境，给予了丰富的课程资源与实验条件，使我能够在理论与实践之间不断探索与进步。感谢各位任课教师的辛勤教学和悉心指导，为我奠定了坚实的专业基础。

同时，我也感谢我的同学与朋友们，在本次毕业设计的过程中，我们互相讨论、彼此鼓励，共同面对项目中出现的难题。你们的支持和陪伴让我在面对挑战时不再孤单。

最后，我要感谢我的家人，是你们一直以来默默的支持与理解，让我能够安心求学，坚定前行。你们是我不断前进的坚强后盾。

本论文凝聚了许多人的关心与帮助，在此谨致以最诚挚的谢意！

参考文献

- [1] 韦衡纳, 郑晓琦. 基于大数据的医疗信息系统平台构建与应用 [J]. 互联网周刊, 2025, (01): 48-50.
- [2] 许书宁. 基于可信执行环境的可搜索加密数据库研究 [D]. 华中科技大学, 2024.
- [3]] 崔津华, 蔡志平, 刘柯江. SGX 隔离技术研究综述 [J]. 华中科技大学学报 (自然科学版), 2024, 52(02): 1-15. DOI: 10.13245/j.hust.240204.
- [4] McKeen F, Alexandrovich I, Anati I, et al. Intel® software guard extensions (intel® sgx) support for dynamic memory management inside an enclave[M]//Proceedings of the Hardware and Architectural Support for Security and Privacy 2016. 2016: 1-9.
- [5] 张锋巍, 周雷, 张一鸣, 等. 可信执行环境: 现状与展望 [J]. 计算机研究与发展, 2024, 61(01): 243-260.
- [6] 刘永志, 秦桂云, 刘蓬涛, 等. 可证明安全的基于 SGX 的公钥认证可搜索加密方案 [J]. 计算机研究与发展, 2023, 60(12): 2709-2724.
- [7] 杨光远, 杨大利, 张羽, 等. 基于可信硬件的隐私数据可搜索加密加速方法研究 [J]. 信息安全研究, 2021, 7(04): 319-327.
- [8] Wu X, Tian D J, Kim C H. Building GPU tees using CPU secure enclaves with gevisor[C]//Proceedings of the 2023 ACM Symposium on Cloud Computing. 2023: 249-264.
- [9] 李旖旎. 基于 Intel SGX 的工业互联网平台数据隐私保护机制研究 [J]. 电脑编程技巧与维护, 2024.
- [10] Ahmed F. Blockchain-Driven Privacy and Security Protection in Decentralized Data Sharing Systems[D]. 中国科学技术大学, 2024. DOI: 10.27517
- [11] 冯朝胜, 罗王平, 秦志光, 等. 支持多种特性的基于属性代理重加密方案 [J]. 通信学报, 2019, 40(06): 177-189.
- [12] Will N C, Maziero C A. Intel Software Guard Extensions Applications: A Survey[J]. ACM Computing Surveys, 2023, 55(14s): 1-38.
- [13] Wang Z, Zhou Y. Analysis and Evaluation of Intel Software Guard Extension-Based Trusted Execution Environment Usage in Edge Intelligence and Internet of Things Scenarios[J]. Future Internet, 2025, 17(1): 32.
- [14] Coppolino L, D'Antonio S, Formicola V, et al. VISE: Combining Intel SGX and Homomorphic Encryption for Cloud Industrial Control Systems[J]. IEEE Transactions on Computers, 2020, 70(5): 711-724.

- [15] Zhou C, Barati M. A TEE-Guarded Data Management System for Time-Scale Data in Industrial Internet of Things[J]. IEEE Internet of Things Journal, 2025.
- [16] Zheng W, Wu Y, Wu X, et al. A Survey of Intel SGX and Its Applications[J]. Frontiers of Computer Science, 2021, 15(3): 153808.
- [17] 赵易. 基于可信执行环境的密态数据库连接查询研究 [D]. 西安电子科技大学,2024.DOI:10.27389/d.cnki.gxadu.2024.002533.
- [18] 赵波, 袁安琪, 安杨.SGX 在可信计算中的应用分析 [J]. 网络与信息安全学报,2021,7(06):126-142.
- [19] 钱文君, 沈晴霓, 吴鹏飞, 等. 大数据计算环境下的隐私保护技术研究进展 [J]. 计算机学报,2022,45(04):669-701.
- [20] 李明煜, 夏虞斌, 陈海波. 面向 SGX2 代新型可信执行环境的内存优化系统 [J]. 软件学报,2022,33(06):2012-2029.DOI:10.13328/j.cnki.jos.006566.
- [21] 孙僖泽, 周福才, 李宇溪, 等. 基于可搜索加密机制的数据库加密方案 [J]. 计算机学报,2021,44(04):806-819.